

Trabajo Práctico Integrador

Taller de Lenguajes de Programación III – React Native

1. Descripción General y Objetivo

El presente Trabajo Práctico Integrador (TPI) tiene como propósito evaluar la capacidad del estudiante para diseñar, codificar e implementar una solución de software orientada a dispositivos móviles empleando el ecosistema moderno de **React Native** y **Expo**.

La entrega posee una naturaleza dual obligatoria: por un lado, se requiere el **desarrollo técnico** de una aplicación móvil completamente funcional y libre de errores; por el otro, la redacción de un **informe teórico-técnico detallado (en formato Word o PDF)** que documente rigurosamente la justificación arquitectónica, el propósito de los componentes utilizados y el análisis del estado de la aplicación.

Resultado de aprendizaje esperado: Al finalizar, el alumno no solo sabrá escribir código reactivo móvil, sino que comprenderá conceptualmente los mecanismos de renderizado nativo, la optimización de memoria en colecciones y el ciclo de vida de los estados en la arquitectura de React.

2. Parte 1: Requisitos Técnicos de la Aplicación

El estudiante deberá optar por desarrollar **una (1)** de las siguientes temáticas sugeridas:

Opción A: Gestor Avanzado de Tareas y Hábitos Diarios

Una aplicación que permita al usuario registrar metas u obligaciones diarias, organizarlas por prioridad, marcarlas como completadas y purgarlas de la vista.

Opción B: Catálogo Personal de Medios (Películas, Libros o Series)

Una biblioteca personalizada donde el usuario registre elementos vistos o leídos, asigne una calificación cuantitativa (estrellas/puntuación), una breve descripción e incluya una URL de imagen de portada.

Componentes obligatorios en el código:

- **Estructura Visual Básica:** Uso estricto de componentes core como `View`, `Text`, e `Image` para estructurar y jerarquizar la interfaz.
- **Captura de Datos:** Implementación de `TextInput` con validación básica (no permitir campos vacíos al registrar un elemento) junto con componentes interactivos (`TouchableOpacity` o `Pressable`).

- **Optimización de Listas:** Uso imperativo de `FlatList` para el renderizado eficiente de los elementos agregados. Queda prohibido el uso de mapeos dinámicos dentro de contenedores scroll estándar para el listado principal.
- **Diseño y Maquetación:** Los estilos deben declararse de manera limpia mediante `StyleSheet.create()` aplicando propiedades de **Flexbox** para asegurar la adaptabilidad de la UI a pantallas de diferentes dimensiones.

3. Parte 2: Estructura del Informe Teórico (Entregable en Word o PDF)

El informe escrito debe ser claro, formal y estructurado de forma secuencial. Servirá para defender el marco conceptual del desarrollo y responderá de manera obligatoria a las siguientes secciones y ejes temáticos:

Sección I: Introducción al Desarrollo Móvil Híbrido

- **React Native:** Definir el framework y explicar detalladamente cómo funciona el renderizado en componentes nativos del sistema operativo, diferenciándolo explícitamente de las soluciones basadas en `WebView` (como Cordova o Ionic antiguas).
- **El ecosistema Expo:** Explicar qué es Expo, cuál es el rol de la aplicación `Expo Go` en el ciclo de testeo rápido y cuáles son las ventajas operativas de la CLI de Expo frente a un flujo de trabajo tradicional nativo (React Native CLI).

Sección II: Anatomía de Componentes Core y Optimización

- **Mapeo de Componentes:** Explicar brevemente con qué etiqueta o estructura clásica del desarrollo Web (HTML) se asocian o equiparan los componentes core utilizados (`View` , `Text` , `TextInput`).
- **Análisis Crítico de Rendimiento:** Justificar teóricamente por qué se eligió un componente `FlatList` en lugar de iterar un arreglo con un `.map()` dentro de un `ScrollView` . Explicar los conceptos de *renderizado perezoso (lazy rendering)* y reciclaje de celdas en memoria.

Sección III: Gestión de Estados e Interactividad

- **Reactividad con `useState`** : Explicar conceptualmente qué es el estado en React. ¿Por qué una mutación de variable tradicional de JavaScript no actualiza la interfaz visual del usuario y qué rol cumple la función set en el re-renderizado del componente?
- **Control de Efectos con `useEffect`** : Describir el funcionamiento del gancho de efecto, el rol de la matriz de dependencias y explicar detalladamente para qué se utilizó en la aplicación desarrollada (ej. inicialización de datos, sincronización o depuración).

Sección IV: Diseño y Flexbox en Dispositivos Móviles

- **Comportamiento del Layout:** Describir las reglas generales de Flexbox aplicadas en React Native.
- **Eje de Dirección por Defecto:** Responder analíticamente: ¿Cuál es la diferencia fundamental entre la propiedad `flexDirection` por defecto en la Web (CSS convencional) y en React Native?

4. Criterios de Evaluación y Rúbrica

Criterio	Puntaje Máx.	Indicadores de Logro
Funcionalidad y Código	25 Puntos	La aplicación compila y ejecuta sin errores en Expo Go. Las funciones de inserción, listado y borrado de elementos operan de manera óptima y fluida.
Arquitectura de UI y Estilos	25 Puntos	Uso correcto de FlatList, control estricto de componentes interactivos y diseño responsive adaptado mediante Flexbox sin solapamiento de componentes.
Precisión y Rigor Teórico	30 Puntos	El informe en Word responde con profundidad técnica cada una de las preguntas planteadas. Demuestra dominio conceptual de React Native y el ciclo de vida.
Articulación y Justificación	20 Puntos	El estudiante es capaz de relacionar la teoría del informe con las líneas de código de su propia autoría, argumentando el porqué de sus decisiones.

Pauta de Entrega Obligatoria: El código fuente debe ser empaquetado comprimido (excluyendo rigurosamente los directorios `node_modules` y `.expo`) o suministrado a través de un repositorio público en GitHub. El informe teórico se adjuntará obligatoriamente como archivo .docx de Microsoft Word o en formato PDF.